

## **CO 3 – ASSESMENT TOOL 1**

**NAME: P.V.N. ANISH BABU**

**REG NO: 192472329**

**COURSE CODE: CSA0924**

**COURSE NAME: JAVA PROGRAMMING**

**SLOT:D**

# MATCHMAKING RACE CONDITION

## *Diagnosing a Check-Then-Act Bug and Redesigning for Safe Concurrent Assignment*

Topic: Thread Synchronization & Communication | Question 29 | Score value: 50

### 1. Scenario

A multiplayer online game's matchmaking service occasionally places the same player into two different matches simultaneously during peak login hours. This corrupts both matches and results in the player being disconnected from one of them with a confusing error. The assignment logic under review is:

```
public void assignToMatch(Player player, Match match) {  
    if (!player.isInMatch()) {  
        match.addPlayer(player);  
        player.setCurrentMatch(match);  
    }  
}
```

### 2. How Two Threads Can Both Win the Race

Two matchmaking worker threads — Thread A evaluating Match 101 and Thread B evaluating Match 204 — can each independently pick the same idle player P as a candidate. The method above performs a classic check-then-act (TOCTOU) sequence with no coordination between the check and the act, and no coordination across the two different Match objects involved:

- Thread A calls `player.isInMatch()` and reads false, because P has not been committed to any match yet.
- Before Thread A finishes, Thread B calls `player.isInMatch()` on the same player object and also reads false — nothing has published a change yet.
- Both threads now believe they independently “won” the right to place P, so both proceed past the guard.
- Thread A executes `match101.addPlayer(P)` and then `player.setCurrentMatch(match101)`.
- Thread B executes `match204.addPlayer(P)` and then `player.setCurrentMatch(match204)`, silently overwriting Thread A's write.

The visible symptom follows directly: P is now rostered in both Match 101 and Match 204, but `P.currentMatch` only ever remembers the last write. Whichever match loses that race still believes it holds the player, drives

game logic for a participant who has already been reassigned elsewhere, and eventually the server disconnects P from the stale match with the “confusing error” described in the prompt.

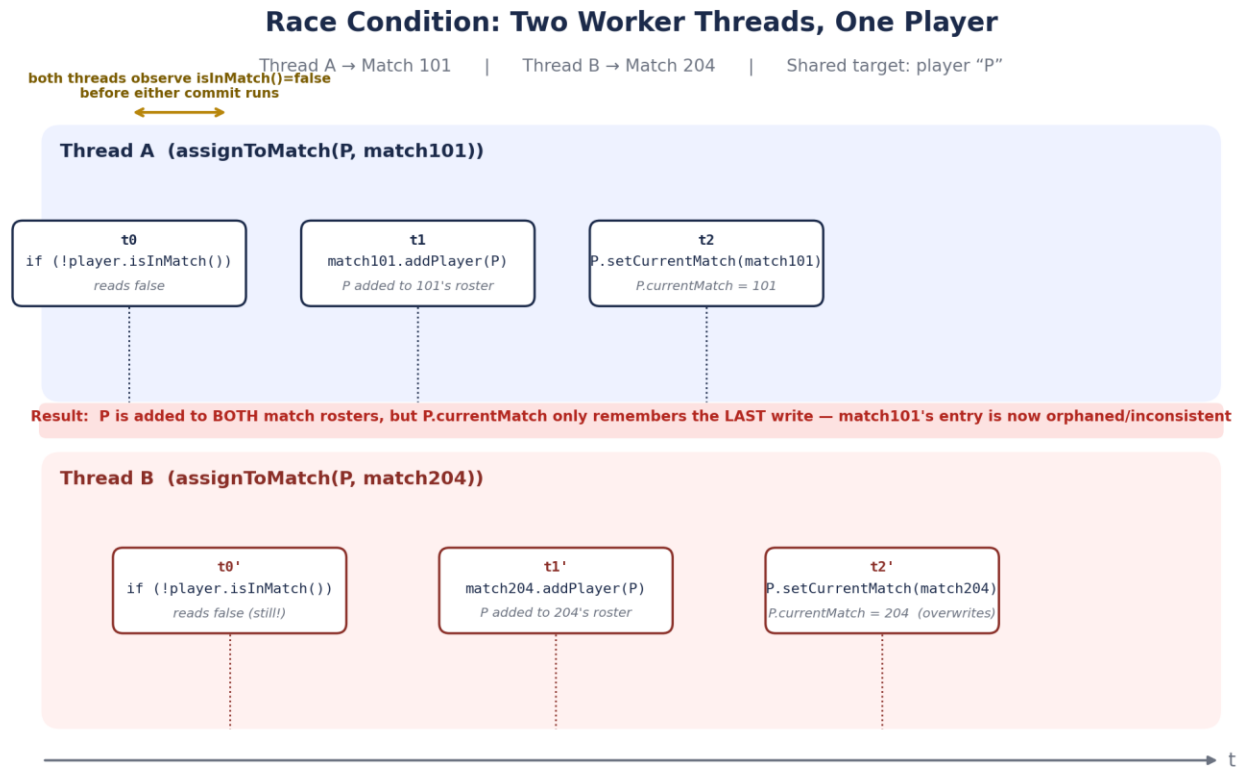


Figure 1 — Interleaving in which both threads pass the guard before either commits, producing a double booking.

### 3. Why Adding “synchronized” to `assignToMatch` Alone Is Not Enough

It is tempting to fix this by simply marking the method synchronized:

```
public synchronized void assignToMatch(Player player, Match match) { ... }
```

This does not close the race, for two independent reasons that both have to be addressed:

#### 3.1 The lock is keyed on the wrong object

An instance method `synchronized` keyword locks on the receiver — in this case the matchmaking service object (or, if written as `synchronized(match)` inside the method, on the `Match` instance). Thread A is operating on `match101` and Thread B is operating on `match204`. If the lock is per-match, the two threads acquire two different locks and never block each other at all; the guard-then-write sequence still interleaves exactly as shown in Figure 1. Locking has to be keyed on the entity that is actually shared between the two threads — the

player — not on the entity that happens to differ per thread. If instead the lock is the single service instance (true method-level synchronized), correctness is restored, but at the cost described in §5.

### 3.2 `match.addPlayer(player)` and `player.setCurrentMatch(match)` are two separate operations, not one

Even with a correctly-keyed lock, the fix must guarantee that the check (`isInMatch`), the match-side write (`addPlayer`), and the player-side write (`setCurrentMatch`) all execute as one atomic unit with respect to any other thread performing the same sequence for the same player. A single synchronized keyword only guarantees this if every thread that can touch this player funnels through the identical lock object for the full duration of all three steps. Two different Match instances give two different monitors, so the “atomic block” that Thread A holds and the “atomic block” that Thread B holds are not mutually exclusive — they are two separate atomic blocks guarding two separate, unrelated locks, and the shared state (the player) is left unprotected between them.

#### Core insight

The contention is between two different Match objects competing for the same Player. Any synchronization strategy that locks on the Match (per-match locking) structurally cannot serialize the two threads against each other, no matter how carefully the critical section inside each match's lock is written.

## 4. Redesigning the Assignment Path

The fix has two parts: move the synchronization point to the player (the actual point of contention), and make the commit itself a single atomic operation so that at most one thread can ever “win” for a given player — with a defined, safe way to back out the losing thread's match-side reservation.

### 4.1 Give every player a single atomic ownership slot

Replace the plain `currentMatch` field with a compare-and-swap-capable reference:

```
private final AtomicReference<Match> currentMatchRef = new AtomicReference<>(null);
```

### 4.2 Reserve → CAS-commit → rollback

Each worker thread now follows a three-step protocol instead of a single unguarded write:

- Reserve: `match.reserveSlot(player)` — a brief, per-match critical section that optimistically increments a candidate counter on that match. This lock is scoped only to the match's own bookkeeping, so it never needs to know about other matches.
- Commit attempt: `player.currentMatchRef.compareAndSet(null, match)` — a single hardware-level atomic instruction. Because every thread targeting this player, regardless of which match it represents, contends on this same reference, exactly one `compareAndSet` can ever observe null and succeed.
- Branch on the result: the winning thread calls `match.confirmPlayer(player)` to make the reservation permanent. Every losing thread calls `match.releaseSlot(player)` to undo its own reservation, then either retries against a different candidate match or reports “player already matched” to its caller.

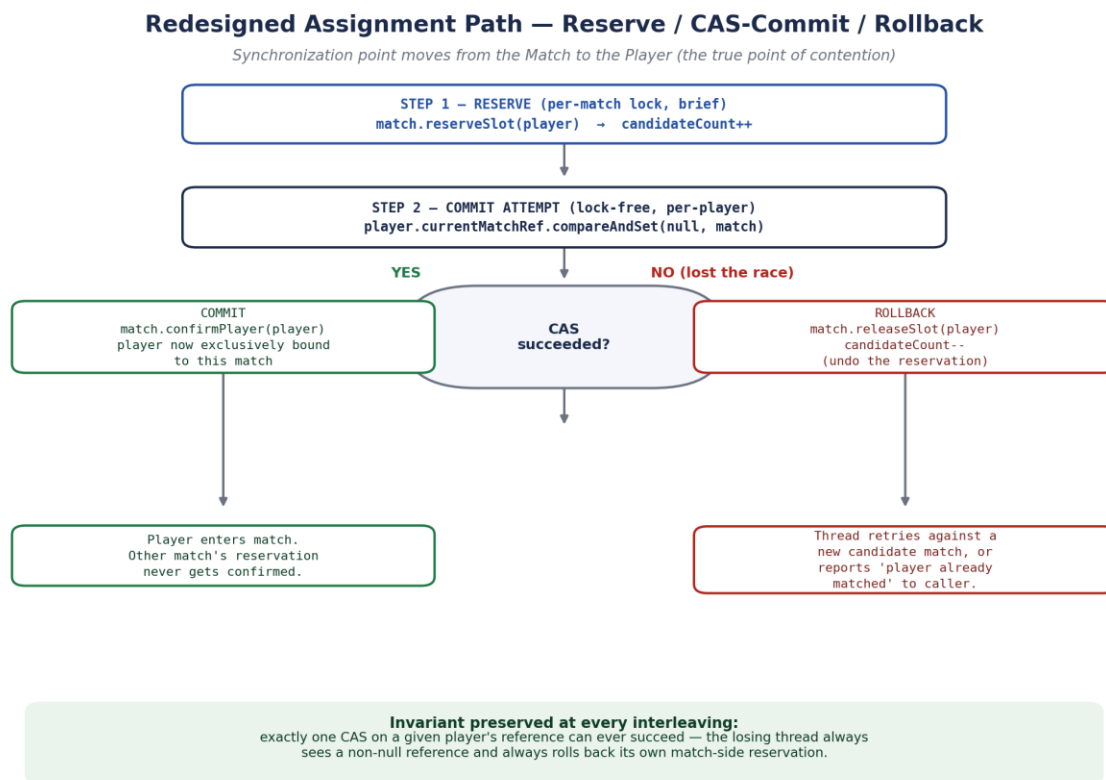


Figure 2 — Reserve / CAS-commit / rollback protocol: the CAS on the player's reference is the single, unavoidable arbitration point.

This closes the race from Figure 1 completely: whichever thread's `compareAndSet` runs first is the only thread that can ever see the reference as null, so it is the only thread that can ever succeed. The other thread's CAS deterministically fails the instant it executes, however the two threads happen to interleave — there is no window in which both can believe they hold the player, because there is no separate check step to race against. The reservation-then-rollback shape also means a match is never left “half-holding” a player who actually

ended up elsewhere: `reserveSlot/releaseSlot` keep the match's own candidate count honest even though the final ownership decision happens somewhere else.

## 5. Justifying the Synchronization Granularity

Three plausible granularities were on the table; the reasoning for picking a lock-free, per-player field over the other two comes down to correctness first and throughput second:

- Whole-service lock (synchronized on `assignToMatch` itself): correct, because every thread funnels through one monitor — but it serializes all assignments across every match in the entire service, even ones involving completely disjoint players. Under peak login hours with many concurrent workers, this collapses matchmaking throughput to that of a single thread.
- Per-match lock (synchronized on the `Match` instance, or inside `match.addPlayer`): scales well because unrelated matches no longer contend — but as shown in §3.1 it does not prevent the actual bug, since the same player can be reserved through two different match locks simultaneously.
- Per-player CAS (`AtomicReference<Match>` plus a brief per-match reservation lock): correct, because the single field that is genuinely shared across the racing threads is the arbitration point, and lock-free because a compare-and-swap needs no monitor acquisition, no blocking, and cannot deadlock against a match-side lock held in the opposite order by another thread.

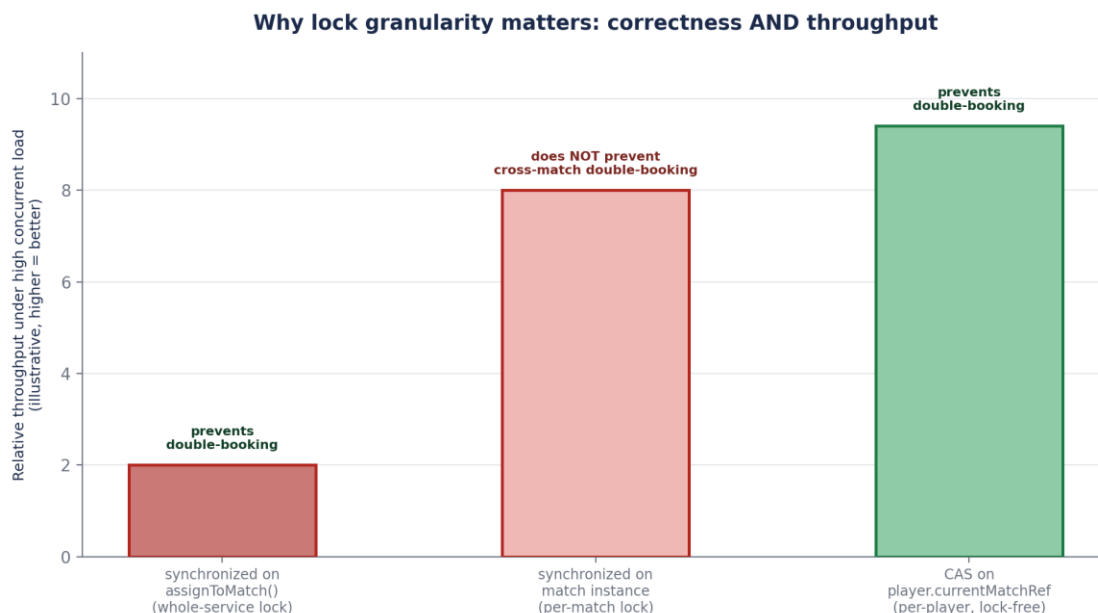


Figure 3 — Illustrative comparison: only locking at the level of the actual contention point (the player) is both correct and scalable.

The per-player CAS also sidesteps a subtler hazard that a naive “lock the player, then lock the match” scheme would introduce: if two threads ever needed to hold a player-lock and a match-lock together, but could acquire them in opposite orders (Thread A: player then match; Thread B: match then player), the service would be exposed to lock-ordering deadlock under exactly the high-concurrency conditions described in the prompt. Reserving on the match without holding any player lock, then resolving ownership with a single atomic instruction on the player, avoids that ordering problem entirely — there is only ever one lock held at a time by any thread.

## 6. Summary

### Result

Two threads can both observe `isInMatch()==false` because the check and the two-step write are not atomic and are not guarded by a shared lock. Synchronizing `assignToMatch()` alone fails whenever the lock is keyed on the Match (which differs per thread) rather than the Player (which is the same for both). Moving to a brief per-match reservation followed by a single `compareAndSet` on the player's match reference makes at most one commit possible, gives the losing thread an explicit rollback path, and keeps synchronization scoped to true points of contention — preserving both correctness and throughput under high concurrent load.